

Technical Design Document (MVP)

WhatsApp Couples Gaming Platform - Tic Tac Toe

Objective

Build a web-based multiplayer gaming platform that can later be integrated into WhatsApp with minimal architectural changes. The first game will be **Tic Tac Toe**, validating the complete multiplayer experience—from room creation to gameplay.

The MVP should prioritize:

- Fast game creation
 - Simple invite flow
 - Real-time multiplayer
 - Mobile-first web experience
 - Future compatibility with WhatsApp Business API
-

Scope (MVP)

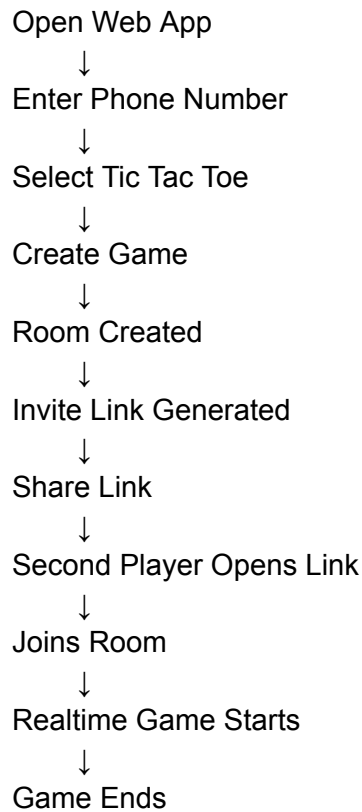
Included

- Mobile-first React web application
- Multiplayer Tic Tac Toe
- Room creation
- Shareable invite links
- Real-time gameplay
- Phone number as the user identifier
- Architecture ready for WhatsApp integration

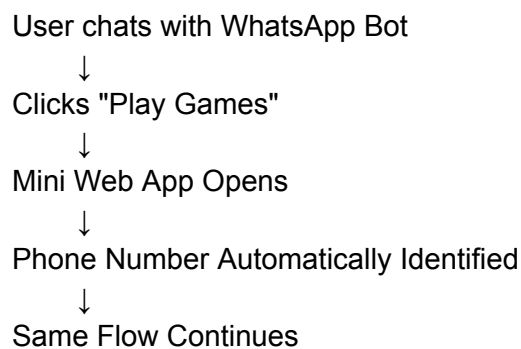
Not Included

- Authentication
- User profiles
- Game history
- Leaderboards
- Payments
- Notifications
- Multiple games (future)

User Flow

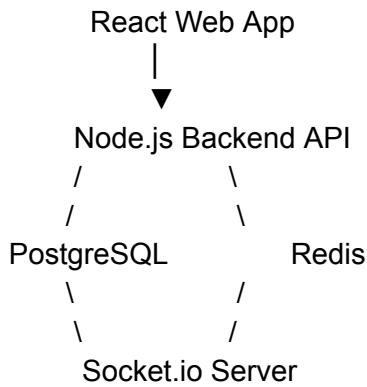


Future WhatsApp Flow



The only change in the future should be the entry point. The gameplay architecture should remain unchanged.

Architecture



Tech Stack

Component	Technology	Why
Frontend	React + TypeScript + Vite	Fast, lightweight, excellent mobile experience
Backend	Node.js + Express	Simple REST APIs and Socket.io integration
Database	PostgreSQL	Reliable permanent storage
Cache	Redis	Fast temporary storage for active games
Realtime	Socket.io	Low-latency multiplayer communication
Deployment	Vercel (Frontend), Railway/Fly.io (Backend)	Simple deployment for MVP

Project Structure

project/

frontend/

backend/

backend/

 routes/

 controllers/

services/
socket/
models/

The backend should remain a **modular monolith**. There is no need for microservices at this stage.

Data Models

User

id
phone_number
created_at

Room

id
game
host_phone
status
created_at
expires_at

Where:

- **id** → Unique room UUID
- **game** → **tic-tac-toe**
- **host_phone** → Phone number of room creator
- **status**
 - WAITING
 - PLAYING
 - FINISHED
 - EXPIRED
- **expires_at** → Prevent stale rooms

No **guest_phone** will be stored in the MVP.

Active Game State (Redis)

Redis stores only active games.

Example:

```
{
  "roomId": "abc123",
  "players": ["+919876543210", "+919123456789"],
  "board": [
    "", "", "",
    "", "", "",
    "", "", "",
    "", "", ""
  ],
  "currentTurn": "X",
  "winner": null
}
```

Once the game finishes, this data can simply be removed.

Backend APIs

Create Room

POST /rooms

Creates a new game room.

Returns:

```
{
  "roomId": "...",
  "inviteLink": "..."
}
```

Join Room

POST /rooms/:id/join

Joins an existing room.

Get Room

GET /rooms/:id

Returns room details.

Start Game

Automatically starts once two players are connected.

Socket Events

Client → Server

join-room

make-move

leave-room

restart-game

Server → Client

player-joined

move-made

game-over

player-left

restart

Invite Flow

Create Room



Generate Room ID



Generate Invite Link

`https://domain.com/join/<room-id>`



Share Link



Second Player Opens Link



Room Validated



Connect via Socket.io



Game Starts

Game Rules

The backend should always validate:

- Correct player's turn
- Valid move
- Cell not occupied
- Winner detection
- Draw detection

The frontend should never be trusted for game logic.

UI Screens

Home

- Enter phone number
 - Play Game
-

Game Selection

- Tic Tac Toe (only game for MVP)
-

Waiting Room

- Invite Link
 - Share Button
 - Waiting for Player...
-

Game Screen

- 3×3 board
 - Current turn
 - Restart game
 - Leave game
-

Result Screen

- Winner
 - Play Again
 - Exit
-

Development Phases

Phase 1 — Foundation

- React project setup
 - Backend setup
 - PostgreSQL
 - Redis
 - Socket.io
-

Phase 2 — Multiplayer

- Room creation
- Invite links

- Join room
 - Waiting room
 - Real-time connection
-

Phase 3 — Gameplay

- Tic Tac Toe engine
 - Move validation
 - Winner detection
 - Draw detection
 - Restart game
-

Phase 4 — Polish

- Mobile responsiveness
 - Animations
 - Better loading states
 - Error handling
 - Room expiration
-

Future Roadmap (Not in MVP)

- WhatsApp Business API integration
 - Automatic phone number identification
 - Multiple games
 - User profiles
 - Match history
 - Achievements
 - Streaks
 - Leaderboards
 - Push notifications
 - AI-powered couple games
-

Engineering Principles

1. Keep the architecture simple.
2. Design everything mobile-first.

3. Optimize for a user reaching a game in under **10 seconds**.
 4. Separate game logic from UI.
 5. Keep the web app independent of WhatsApp so the entry point can change without affecting gameplay.
 6. Avoid premature optimization or microservices until the product is validated.
-

Cursor Implementation Goal

Build a production-quality MVP where two users can:

1. Open the web app.
2. Enter their phone number.
3. Create a Tic Tac Toe room.
4. Share an invite link.
5. Join from another device.
6. Play in real time with Socket.io.
7. Handle reconnects gracefully if a player refreshes the page.
8. End the game and clean up the room.

The codebase should be clean, modular, and extensible so that adding future games (Connect 4, Chess, Truth or Dare, etc.) only requires implementing new game logic while reusing the same room, invite, and multiplayer infrastructure.